Laboratorio 1: Estandarización del Entorno de Programación Móvil 2026

Este laboratorio es obligatorio para asegurar que todos los estudiantes y estaciones de trabajo operen bajo una arquitectura homogénea, minimizando errores de compatibilidad en los módulos de **Jetpack Compose** y **Edge AI**.

1. Configuración de Hardware y BIOS (Paso Crítico)

Antes de instalar software, se debe preparar el hardware para la virtualización.

1. Reinicie la PC e ingrese a la **BIOS/UEFI** (teclas F2, F10, Del).
2. Busque la opción **Virtualization Technology** (VT-x en Intel o AMD-V en AMD) y cámbiela a **Enabled**.
3. Guarde los cambios y reinicie. **Sin esto, el emulador no funcionará**.

2. Instalación de Herramientas Base (Versiones 2026)

Para mantener la homogeneidad, instale exactamente estas versiones:

- **IDE:** Android Studio **Ladybug (2024.2.1)** o superior desde el *Download Archive* de Android (asegurarse de estar en la página web en inglés).
- **SDK:** Desde el SDK Manager, instale la **API 36 (Android 16).**
- **JDK:** No instale Java externamente. Usaremos el **JBR 21** (JetBrains Runtime) que viene embebido en Android Studio.

3. Configuración del SDK Manager

Dentro de Android Studio, vaya a Settings > Languages & Frameworks > Android SDK e instale:

- **SDK Platform:** Android 16.0 (API Level 36).
- **SDK Tools:**
    - Android SDK Build-Tools 36.0.0.
    - Android Emulator.
    - Android SDK Platform-Tools.

4. Creación del Dispositivo Virtual (AVD) Homogéneo

Para que las pruebas de rendimiento y UI sean consistentes:

1. Abra el **Device Manager** (Ícono  en el menú de la derecha de la pantalla principal) y haga clic en **Create Device** (símbolo +).
2. **Hardware:** Seleccione **Pixel 8**.
3. **System Image:** Seleccione **Android 16.0 (API 36)** con la variante **"Google Play"**.
4. **Configuración Avanzada:** Asigne al menos **2048 MB (2GB)** de RAM al emulador para evitar cierres inesperados.

---

5. IDENTIFICACIÓN DE ARCHIVOS (¡IMPORTANTE!)

Antes de editar, identifique los dos archivos de configuración. En el panel izquierdo (vista Android), despliegue Gradle Scripts:

- **build.gradle.kts (Project: MyApplication)**: Es el archivo **RAÍZ**. Aquí se definen las reglas globales y versiones de plugins para todo el semestre.
- **build.gradle.kts (Module :app)**: Es el archivo del **MÓDULO**. Aquí se configura la aplicación específica, sus permisos y sus librerías de interfaz.

6. Bloqueo de Versiones mediante Gradle (Código)

Para evitar que el proyecto se actualice solo y rompa las prácticas, utilizaremos el **Kotlin DSL** y el **Gradle Wrapper**. Seguir los siguientes pasos:

A. El Inventario (Gradle Scripts/libs.versions.toml)

Edite este archivo para bloquear las versiones industriales de 2026. De ser necesario agregue las siguientes instrucciones o, si ya existen estas líneas, asegúrese que los valores coincidan con el siguiente código:

```
[versions]
agp = "8.7.0"
kotlin = "2.0.0"

[plugins]
android-application = { id = "com.android.application", version.ref = "agp"
}
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref =
"kotlin" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref =
"kotlin" }
```

B. Configuración Global (Archivo RAÍZ / Project)

Abra **build.gradle.kts (Project: ...)** y use solo los alias. No escriba números de versión aquí:

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
}
```

C. Forzar versión de Java y SDK

En el archivo **build.gradle.kts (Module :app)**, reemplace o asegure los siguientes valores para "congelar" las versiones de compilación:

```
android {
    // Bloqueamos a la API 36 definida para el curso
    compileSdk = 36

    defaultConfig {

        // MinSdk 26 asegura compatibilidad con el 90% del mercado actual
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
    }

    compileOptions {
        // Forzamos Java 21 para evitar errores de bytecode
        sourceCompatibility = JavaVersion.VERSION_21
        targetCompatibility = JavaVersion.VERSION_21
    }

    kotlinOptions {
        jvmTarget = "21"
    }

    buildFeatures {
        compose = true
    }
}
```

D. Blindaje de Terminal (gradle.properties)

Para que la terminal no use versiones de Java externas (como Java 25), agregue al final de este archivo. Asegúrese de usar el **path** correcto que corresponda con el path donde instaló Android Studio en su máquina, y utilice la barra \ y doble barras \\ como se indica en el ejemplo:

```
org.gradle.java.home=C\:\\Program Files\\Android\\Android Studio\\jbr
```

7. Verificación de configuraciones

1. Vaya a **File > Settings > Build, Execution, Deployment > Build Tools > Gradle** y asegúrese de que **Gradle JDK** sea **jbr-21**.
2. Haga clic en el icono del elefante (**Sync Project**).
3. En la **Terminal** inferior, ejecute: **./gradlew --version**

REQUISITO DE APROBACIÓN: El comando debe devolver Launcher JVM: 21.

Si después de esto sigue apareciendo Java 25 u otra versión diferente a la 21, es porque el comando **./gradlew** está detectando la variable **JAVA_HOME** del Windows de la PC antes que la del proyecto. Sin embargo, al configurar el archivo **gradle.properties**, el proceso de compilación real usará Java 21, que es lo que nos importa para evitar los errores de **"Application Not Responding" (ANR).**

8. Resultados Esperados

Un laboratorio se considera exitoso si el estudiante cumple estos cuatro hitos:

4. **Build Successful:** El proyecto compila sin errores de descarga de dependencias.
5. **Emulador Operativo:** El Pixel 8 virtual inicia y muestra el Home de Android.
6. **Null Safety:** El código Kotlin no muestra advertencias de interoperabilidad.
7. **Git Init:** Se ha realizado el primer *commit* local incluyendo el archivo .gitignore generado por Android Studio.

Tip: Si su PC cuenta con poca RAM, agregue la línea **org.gradle.jvmargs=-Xmx2048m** en el archivo **gradle.properties** para limitar el consumo de memoria de Gradle y evitar bloqueos del sistema.

8. Entregables:

1. **Código Fuente:** Proyecto de NetBeans completo y comprimido en formato .zip o .rar. (si aplica)
2. **El Archivo .JAR:** El programa ejecutable generado en la carpeta dist. (si aplica)
3. **Informe (PDF):** Documento breve con capturas de pantalla que muestren:
    - La correcta configuración del sistema.

- Build Successful.
- Emulador Operativo.

Notas

- No olvide revisar la rúbrica de evaluación para complementar su informe.

ANEXO

A partir de este momento, este proyecto se usará como **Plantilla Maestra (Template)**. No es necesario repetir toda la configuración manual desde cero para cada nuevo laboratorio del semestre.

1. ¿Por qué es una buena práctica de ingeniería?

- **Consistencia Técnica:** Asegura que todos los proyectos tengan las mismas versiones de Java 21, Kotlin 2.0 y API 36, evitando errores que solo aparecen al final del desarrollo.
- **Eficiencia (Time-to-Market):** Reduce el tiempo de configuración inicial de 20 minutos a solo 2 minutos, permitiendo enfocarse en la lógica de programación y no en la carpintería de archivos.
- **Portabilidad:** Si la plantilla funciona, cualquier proyecto derivado de ella también funcionará para presentarlo como tarea o cuando se suba a GitHub.

2. Cómo "Clonar" la plantilla para un nuevo Laboratorio

Cuando se desee iniciar el **Laboratorio 2, 3, etc.**, no usar "New Project". En su lugar, seguir este procedimiento:

1. **Copia Física:** En el explorador de archivos (Windows), copiar la carpeta completa del Laboratorio 1 y pegarla con el nuevo nombre (ej. Laboratorio2_NombreApellido).
2. **Limpieza de Caché:** Borrar las carpetas ocultas .gradle, .idea y build dentro de la nueva carpeta (esto obliga a Android Studio a regenerar los índices limpios).
3. **Renombrado del Proyecto:** Abrir la nueva carpeta en Android Studio.
    - Ir al archivo **settings.gradle.kts** y cambiar la línea **rootProject.name = "Laboratorio0"** por el nombre del nuevo laboratorio.
4. **Sincronización:** Hacer clic en el icono del elefante (**Sync Project with Gradle Files**).
5. **Verificación Final:** Abrir la terminal y ejecutar ./gradlew --version para confirmar que el blindaje (Java 21) se mantuvo en la copia.

Nota: Si realiza un cambio en el **libs.versions.toml** de la plantilla (por ejemplo, cuando en la **Semana 12** agreguemos la librería **Retrofit**), recuerde actualizar también sus proyectos anteriores si desea que compartan las mismas herramientas.